Assignment 5B: Elo Calculations

Mohd Afwan Shaikh

2026-10-02

Contents

Introduction and Approach 1
Data Collection and Preparation 1
Extracting Opponents and Game Results 3
Matching Players and Opponent Ratings 4
Calculating Elo Expected Scores 6
Identifying Overperforming and Underperforming Players 8
Visualizing Player Performance 9
Relationship Between Pre-Tournament Rating and Performance 10
Findings and Discussion 12
Conclusion 12
References 13

Introduction and Approach

For this assignment, I continued working with the chess tournament data from Project 1. I used each player's pre-tournament rating and their individual opponents' ratings to calculate expected tournament scores using the Elo expected-score formula.

I imported the original tournament data into R, extracted the opponents and results for each round, and matched each player with their pre-tournament rating. I then calculated the expected score for every played game and added these values to obtain each player's total expected score.

Next, I compared each player's expected score with their actual score and calculated the difference. Finally, I identified the five players who most overperformed and the five who most underperformed, presenting the results in tables and a visualization.

Data Collection and Preparation

I used the original chess tournament data from Project 1. The original text file contains each player's pre-tournament rating, tournament score, and opponents for each round. I also imported my cleaned dataset to verify the player information.

```r
# Import the cleaned tournament data
chess_data <- read.csv("chess_tournament_clean.csv")

# Read the original tournament file
tournament_lines <- readLines("tournamentinfo.txt")
```

```
## Warning in readLines("tournamentinfo.txt"): incomplete final line found on
## 'tournamentinfo.txt'
```

```r
# Inspect both datasets
head(chess_data)
```

```
##           Player_Name State Total_Points Pre_Rating Avg_Opponent_Rating
## 1            GARY HUA    ON          6.0       1794                1605
## 2     DAKSHESH DARURI    MI          6.0       1553                1469
## 3        ADITYA BAJAJ    MI          6.0       1384                1564
## 4 PATRICK H SCHILLING    MI          5.5       1716                1574
## 5          HANSHI ZUO    MI          5.5       1655                1501
## 6         HANSEN SONG    OH          5.0       1686                1519
```

```r
names(chess_data)
```

```
## [1] "Player_Name"         "State"               "Total_Points"
## [4] "Pre_Rating"          "Avg_Opponent_Rating"
```

```r
dim(chess_data)
```

```
## [1] 64  5
```

```r
# Display the first 15 lines of the original file
cat(tournament_lines[1:15], sep = "\n")
```

```
## -----------------------------------------------------------------------------------------
##  Pair | Player Name                     |Total|Round|Round|Round|Round|Round|Round|Round|
##  Num  | USCF ID / Rtg (Pre->Post)       | Pts |  1  |  2  |  3  |  4  |  5  |  6  |  7  |
## -----------------------------------------------------------------------------------------
##    1  | GARY HUA                        |6.0  |W  39|W  21|W  18|W  14|W   7|D  12|D   4|
##   ON  | 15445895 / R: 1794   ->1817     |N:2  |W    |B    |W    |B    |W    |B    |W    |
## -----------------------------------------------------------------------------------------
##    2  | DAKSHESH DARURI                 |6.0  |W  63|W  58|L   4|W  17|W  16|W  20|W   7|
##   MI  | 14598900 / R: 1553   ->1663     |N:2  |B    |W    |B    |W    |B    |W    |B    |
## -----------------------------------------------------------------------------------------
##    3  | ADITYA BAJAJ                    |6.0  |L   8|W  61|W  25|W  21|W  11|W  13|W  12|
##   MI  | 14959604 / R: 1384   ->1640     |N:2  |W    |B    |W    |B    |W    |B    |W    |
## -----------------------------------------------------------------------------------------
##    4  | PATRICK H SCHILLING             |5.5  |W  23|D  28|W   2|W  26|D   5|W  19|D   1|
##   MI  | 12616049 / R: 1716   ->1744     |N:2  |W    |B    |W    |B    |W    |B    |B    |
```

Extracting Opponents and Game Results

I extracted each player's opponents and round results from the original tournament file. These individual results allowed me to calculate an expected score for every played game using the Elo formula.

```
# Identify player result lines (the lines beginning with a player number)
player_lines <- tournament_lines[
  grepl("^\\s*[0-9]+\\s*\\|", tournament_lines)
]

# Extract the seven round-result fields for each player
round_data <- lapply(player_lines, function(line) {
  fields <- strsplit(line, "\\|")[[1]]
  fields <- trimws(fields)

  data.frame(
    Player_ID = as.integer(fields[1]),
    Round = 1:7,
    Result = fields[4:10]
  )
})

round_data <- bind_rows(round_data)

# Separate game result (W, L, D) from opponent number
round_data <- round_data %>%
  mutate(
    Game_Result = str_extract(Result, "^[WLD]"),
    Opponent_ID = as.integer(str_extract(Result, "[0-9]+$")),
    Actual_Game_Score = case_when(
      Game_Result == "W" ~ 1,
      Game_Result == "D" ~ 0.5,
      Game_Result == "L" ~ 0,
      TRUE ~ NA_real_
    )
  )

# Inspect the first player's seven rounds
print(round_data %>% filter(Player_ID == 1))
```

```
##   Player_ID Round Result Game_Result Opponent_ID Actual_Game_Score
## 1         1     1  W  39           W          39               1.0
## 2         1     2  W  21           W          21               1.0
## 3         1     3  W  18           W          18               1.0
## 4         1     4  W  14           W          14               1.0
## 5         1     5  W   7           W           7               1.0
## 6         1     6  D  12           D          12               0.5
## 7         1     7  D   4           D           4               0.5
```

```
# Check the total number of players and rounds
n_distinct(round_data$Player_ID)
```

```
## [1] 64
```

```r
nrow(round_data)
```

```
## [1] 448
```

```r
# Identify rounds without a recorded opponent
print(round_data %>% filter(is.na(Opponent_ID)))
```

```
##    Player_ID Round Result Game_Result Opponent_ID Actual_Game_Score
## 1         12     5      H        <NA>          NA                NA
## 2         16     3      H        <NA>          NA                NA
## 3         16     7      U        <NA>          NA                NA
## 4         22     5      H        <NA>          NA                NA
## 5         27     7      U        <NA>          NA                NA
## 6         29     7      U        <NA>          NA                NA
## 7         36     5      H        <NA>          NA                NA
## 8         37     1      B        <NA>          NA                NA
## 9         37     5      H        <NA>          NA                NA
## 10        38     5      H        <NA>          NA                NA
## 11        41     5      X        <NA>          NA                NA
## 12        41     6      U        <NA>          NA                NA
## 13        41     7      U        <NA>          NA                NA
## 14        44     1      B        <NA>          NA                NA
## 15        48     3      H        <NA>          NA                NA
## 16        48     5      H        <NA>          NA                NA
## 17        49     6      H        <NA>          NA                NA
## 18        49     7      U        <NA>          NA                NA
## 19        50     5      H        <NA>          NA                NA
## 20        53     1      H        <NA>          NA                NA
## 21        53     3      H        <NA>          NA                NA
## 22        53     5      U        <NA>          NA                NA
## 23        53     7      U        <NA>          NA                NA
## 24        54     4      B        <NA>          NA                NA
## 25        55     5      B        <NA>          NA                NA
## 26        56     1      H        <NA>          NA                NA
## 27        56     5      H        <NA>          NA                NA
## 28        57     7      B        <NA>          NA                NA
## 29        58     6      B        <NA>          NA                NA
## 30        59     2      B        <NA>          NA                NA
## 31        60     6      H        <NA>          NA                NA
## 32        60     7      U        <NA>          NA                NA
## 33        62     2      U        <NA>          NA                NA
## 34        62     3      U        <NA>          NA                NA
## 35        62     4      U        <NA>          NA                NA
## 36        62     5      U        <NA>          NA                NA
## 37        62     6      U        <NA>          NA                NA
## 38        62     7      U        <NA>          NA                NA
## 39        63     6      H        <NA>          NA                NA
## 40        63     7      U        <NA>          NA                NA
```

Matching Players and Opponent Ratings

I matched each player and their opponents to their pre-tournament ratings. I checked that the player names were in the same order in the original and cleaned datasets before assigning player numbers.

```r
# Extract player names from the original tournament file
original_players <- data.frame(
  Player_ID = as.integer(
    trimws(sapply(strsplit(player_lines, "\\|"), `[`, 1))
  ),
  Original_Name = trimws(
    sapply(strsplit(player_lines, "\\|"), `[`, 2)
  )
)

# Compare original names with the cleaned CSV
name_check <- original_players %>%
  mutate(
    Cleaned_Name = chess_data$Player_Name,
    Names_Match = toupper(trimws(Original_Name)) ==
                  toupper(trimws(Cleaned_Name))
  )

print(name_check %>% filter(!Names_Match))
```

```
## [1] Player_ID     Original_Name Cleaned_Name  Names_Match
## <0 rows> (or 0-length row.names)
```

```r
# Stop if the player order does not match
stopifnot(all(name_check$Names_Match))

# Add player numbers to the cleaned dataset
players <- chess_data %>%
  mutate(Player_ID = original_players$Player_ID)

# Attach each player's and opponent's rating
games <- round_data %>%
  left_join(
    players %>% select(Player_ID, Player_Name, Pre_Rating),
    by = "Player_ID"
  ) %>%
  left_join(
    players %>%
      select(
        Opponent_ID = Player_ID,
        Opponent_Rating = Pre_Rating
      ),
    by = "Opponent_ID"
  )

# Inspect Gary Hua's games
print(games %>% filter(Player_ID == 1))
```

```
##   Player_ID Round Result Game_Result Opponent_ID Actual_Game_Score Player_Name
## 1         1     1  W  39           W          39               1.0    GARY HUA
## 2         1     2  W  21           W          21               1.0    GARY HUA
## 3         1     3  W  18           W          18               1.0    GARY HUA
## 4         1     4  W  14           W          14               1.0    GARY HUA
```

```
## 5          1     5  W   7          W           7               1.0    GARY HUA
## 6          1     6  D  12          D          12               0.5    GARY HUA
## 7          1     7  D   4          D           4               0.5    GARY HUA
##   Pre_Rating Opponent_Rating
## 1       1794            1436
## 2       1794            1563
## 3       1794            1600
## 4       1794            1610
## 5       1794            1649
## 6       1794            1663
## 7       1794            1716
```

Elo Expected-Score Formula

The expected score for player A against player B is:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

Here, R_A is the player's pre-tournament rating, R_B is the opponent's pre-tournament rating, and E_A is the expected score for that game.

I used this formula to calculate the expected score for each played game, then summed the results for each player.

Calculating Elo Expected Scores

I calculated each player's expected score against every opponent using the Elo expected-score formula. I then added the expected scores across all played games and compared them with the points the player actually earned in those games. Byes, forfeits, and unplayed rounds were excluded because they do not have an opponent rating.

```
# Calculate expected scores for games with an opponent
elo_games <- games %>%
  filter(
    !is.na(Opponent_ID),
    !is.na(Pre_Rating),
    !is.na(Opponent_Rating),
    !is.na(Actual_Game_Score)
  ) %>%
  mutate(
    Expected_Game_Score = 1 /
      (1 + 10^((Opponent_Rating - Pre_Rating) / 400))
  )

# Add expected and actual scores for each player
elo_results <- elo_games %>%
  group_by(Player_ID, Player_Name, Pre_Rating) %>%
  summarise(
    Games_Played = n(),
    Expected_Score = sum(Expected_Game_Score),
    Actual_Score = sum(Actual_Game_Score),
    .groups = "drop"
```

```
  ) %>%
  mutate(
    Performance_Difference = Actual_Score - Expected_Score
  ) %>%
  arrange(desc(Performance_Difference))

# Display Gary Hua's individual games
print(
  elo_games %>%
    filter(Player_ID == 1) %>%
    select(
      Round,
      Opponent_ID,
      Opponent_Rating,
      Actual_Game_Score,
      Expected_Game_Score
    )
)
```

```
##   Round Opponent_ID Opponent_Rating Actual_Game_Score Expected_Game_Score
## 1     1          39            1436                1.0           0.8870357
## 2     2          21            1563                1.0           0.7907981
## 3     3          18            1600                1.0           0.7533861
## 4     4          14            1610                1.0           0.7425356
## 5     5           7            1649                1.0           0.6973451
## 6     6          12            1663                0.5           0.6800707
## 7     7           4            1716                0.5           0.6104024
```

```
# Display Gary Hua's tournament results
print(elo_results %>% filter(Player_ID == 1))
```

```
## # A tibble: 1 x 7
##   Player_ID Player_Name Pre_Rating Games_Played Expected_Score Actual_Score
##       <int> <chr>            <int>        <int>          <dbl>        <dbl>
## 1         1 GARY HUA          1794            7           5.16            6
## # i 1 more variable: Performance_Difference <dbl>
```

```
# Validate the number of records and players
stopifnot(nrow(elo_games) == 408)
stopifnot(nrow(elo_results) == 64)

cat("Elo calculations validated successfully.\n")
```

```
## Elo calculations validated successfully.
```

The dataset contained 64 players and seven rounds, producing 448 player-round records. Of these, 40 did not have an identified opponent. I excluded those records from the Elo calculations, leaving 408 player-game records representing 204 played games.

The Elo rating system estimates a player's expected score using the difference between two players' ratings. A player rated 400 points higher than an opponent has expected-score odds of 10 to 1. A win is worth one point, a draw is worth half a point, and a loss is worth zero points. I used the expected-score formula explained in *A Comprehensive Guide to Chess Ratings* by Mark E. Glickman to calculate each player's expected performance.

Identifying Overperforming and Underperforming Players

I ranked the players by subtracting their expected scores from their actual scores. A positive difference indicates that a player performed better than expected, while a negative difference indicates that they performed worse than expected.

```
# Round values for presentation without changing the original calculations
ranked_players <- elo_results %>%
  mutate(
    Expected_Score = round(Expected_Score, 2),
    Actual_Score = round(Actual_Score, 2),
    Performance_Difference = round(Performance_Difference, 2)
  )

# Five players who performed most above expectations
top_overperformers <- ranked_players %>%
  arrange(desc(Performance_Difference)) %>%
  slice_head(n = 5)

# Five players who performed most below expectations
top_underperformers <- ranked_players %>%
  arrange(Performance_Difference) %>%
  slice_head(n = 5)

print(
  knitr::kable(
    top_overperformers %>%
      select(
        Player_Name,
        Pre_Rating,
        Games_Played,
        Expected_Score,
        Actual_Score,
        Performance_Difference
      ),
    caption = "Five Most Overperforming Players"
  )
)
```

Table 1: Five Most Overperforming Players

Player_Name	Pre_Rating	Games_Played	Expected_Score	Actual_Score	Performance_Difference
ADITYA BAJAJ	1384	7	1.95	6.0	4.05
ZACHARY JAMES HOUGHTON	1220	7	1.37	4.5	3.13
ANVIT RAO	1365	7	1.94	5.0	3.06
JACOB ALEXANDER LAVALLEY	377	7	0.04	3.0	2.96
STEFANO LEE	1411	7	2.29	5.0	2.71

```
cat("\n\n")
```

```
print(
  knitr::kable(
    top_underperformers %>%
      select(
        Player_Name,
        Pre_Rating,
        Games_Played,
        Expected_Score,
        Actual_Score,
        Performance_Difference
      ),
    caption = "Five Most Underperforming Players"
  )
)
```

Table 2: Five Most Underperforming Players

Player_Name	Pre_Rating	Games_Played	Expected_Score	Actual_Score	Performance_Difference
LOREN SCHWIEBERT	1745	7	6.28	3.5	-2.78
GEORGE AVERY JONES	1522	7	6.02	3.5	-2.52
LARRY HODGE	1270	6	3.40	1.0	-2.40
JARED GE	1332	7	5.01	3.0	-2.01
RISHI SHETTY	1494	7	5.09	3.5	-1.59

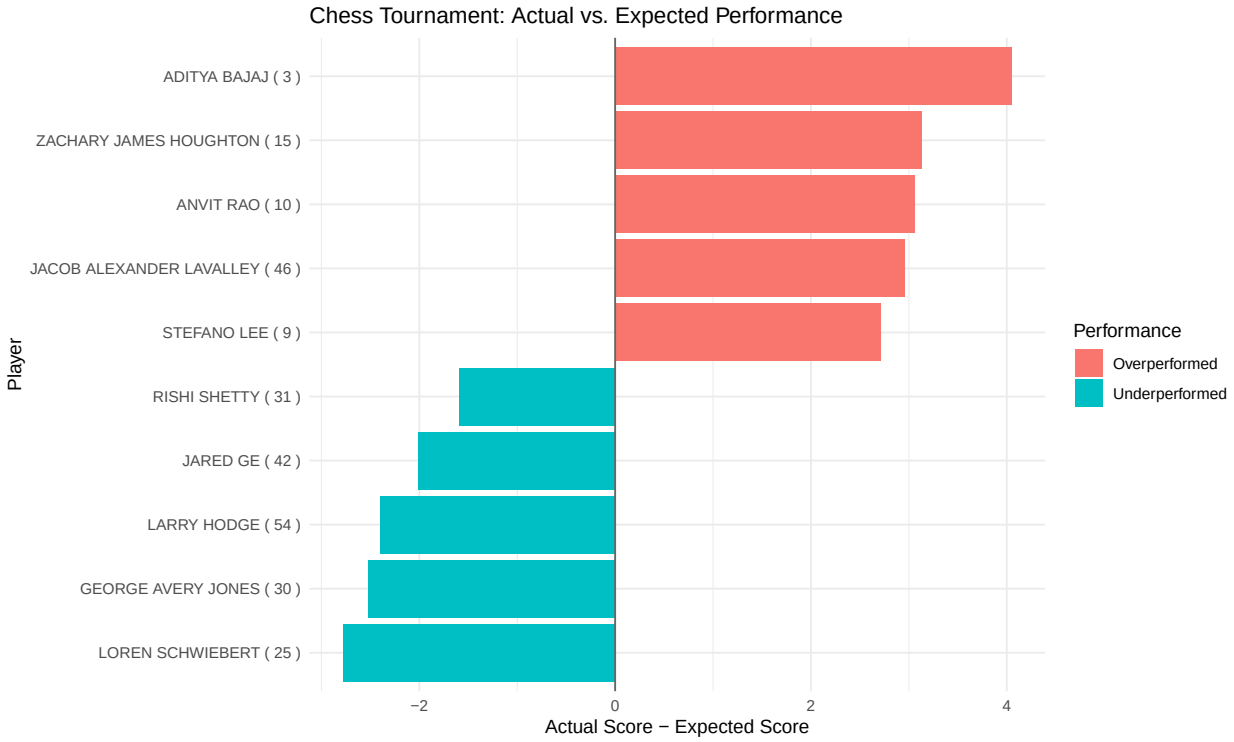
Visualizing Player Performance

The following chart compares the players who performed furthest above and below their expected scores.

```
performance_chart <- bind_rows(
  top_overperformers %>% mutate(Group = "Overperformed"),
  top_underperformers %>% mutate(Group = "Underperformed")
) %>%
  mutate(
    Player_Label = paste(Player_Name, "(", Player_ID, ")")
  )

ggplot(
  performance_chart,
  aes(
    x = reorder(Player_Label, Performance_Difference),
    y = Performance_Difference,
    fill = Group
  )
) +
  geom_col() +
  geom_hline(yintercept = 0, color = "gray40") +
  coord_flip() +
  labs(
    title = "Chess Tournament: Actual vs. Expected Performance",
```

```
    x = "Player",
    y = "Actual Score - Expected Score",
    fill = "Performance"
  ) +
  theme_minimal()
```



Relationship Between Pre-Tournament Rating and Performance

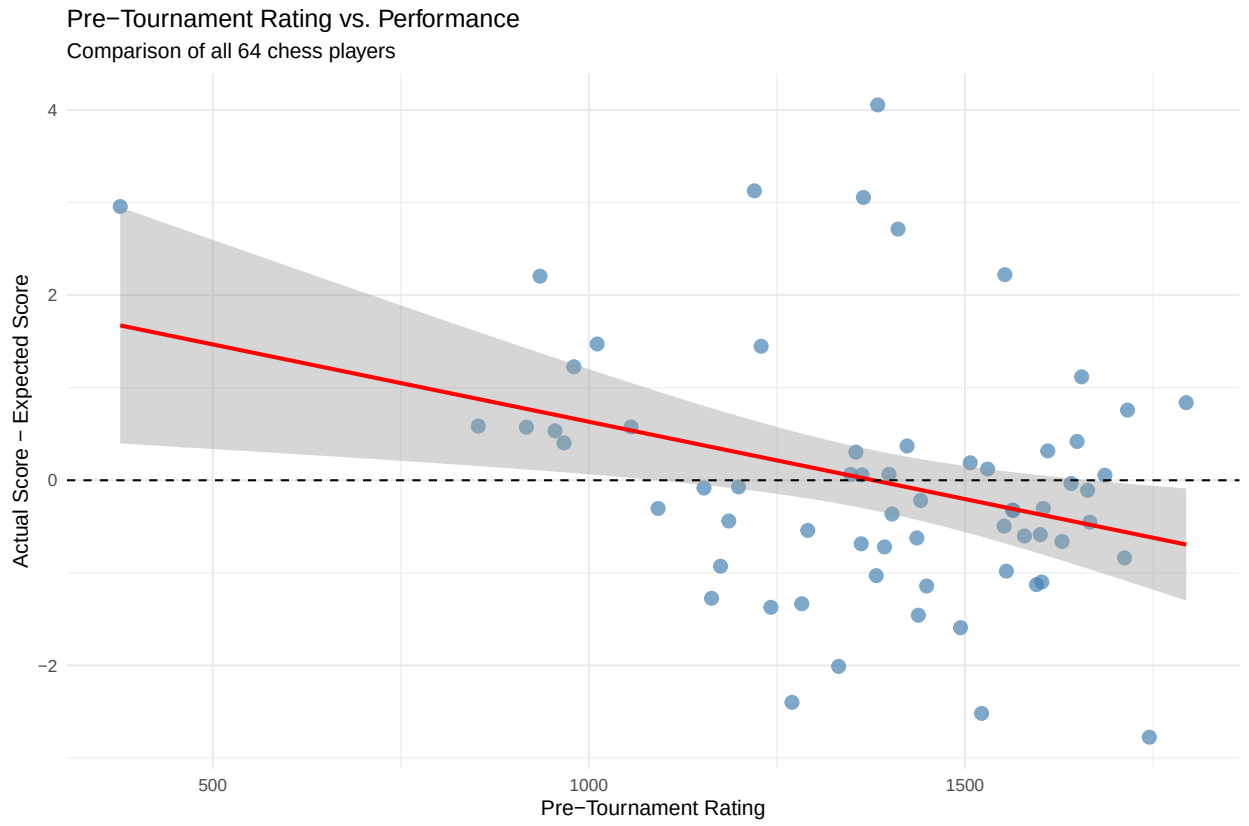
I wanted to investigate whether players with higher pre-tournament ratings were more likely to perform above expectations. To explore this relationship, I compared each player's pre-tournament rating with the difference between their actual and expected scores.

A positive performance difference indicates that a player exceeded expectations, while a negative difference indicates that they performed below expectations.

```
ggplot(
  elo_results,
  aes(
    x = Pre_Rating,
    y = Performance_Difference
  )
) +
  geom_point(
    color = "steelblue",
    size = 3,
    alpha = 0.7
  ) +
  geom_smooth(
```

```
    method = "lm",
    se = TRUE,
    color = "red"
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
  labs(
    title = "Pre-Tournament Rating vs. Performance",
    subtitle = "Comparison of all 64 chess players",
    x = "Pre-Tournament Rating",
    y = "Actual Score - Expected Score"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
correlation <- cor(
  elo_results$Pre_Rating,
  elo_results$Performance_Difference,
  use = "complete.obs"
)

cat(
```

```
  "Correlation between pre-tournament rating and performance:",
  round(correlation, 3)
)
```

```
## Correlation between pre-tournament rating and performance: -0.325
```

Relationship Between Rating and Performance

The correlation between pre-tournament rating and performance difference was **−0.325**, indicating a weak-to-moderate negative relationship.

The scatter plot shows a downward trend, suggesting that lower-rated players generally performed better relative to their expected scores, while higher-rated players were somewhat more likely to perform below expectations.

However, the points are widely scattered, indicating that pre-tournament rating alone does not strongly predict whether a player will exceed expectations. This relationship describes the 64 players in this tournament and does not necessarily apply to other tournaments.

Findings and Discussion

Players Who Overperformed

Aditya Bajaj had the largest positive difference between his actual and expected scores. Zachary James Houghton, Anvit Rao, Jacob Alexander Lavalley, and Stefano Lee were the other four highest overperformers.

These players earned more points than predicted by their pre-tournament ratings and the ratings of their opponents.

Players Who Underperformed

Loren Schwiebert had the largest negative performance difference. His expected score was 6.28 points, but he earned only 3.50 points in played games, a difference of −2.78 points.

George Avery Jones was next, with a difference of −2.52 points, followed by Larry Hodge (−2.40), Jared Ge (−2.01), and Rishi Shetty (−1.59).

Interpreting the Results

The Elo expected-score formula estimates a player's expected performance based on the difference between their rating and their opponent's rating. It does not guarantee the outcome of an individual game.

For this analysis, I excluded byes, forfeits, and unplayed rounds from both expected and actual scores. This allowed me to compare the points earned in played games with the points predicted for those same games.

Conclusion

Using the Elo expected-score formula, I calculated expected scores for all 64 players in the chess tournament and compared them with their actual results. I identified the five players who most exceeded expectations and the five players who fell furthest below expectations.

I also examined the relationship between pre-tournament rating and performance difference. The correlation was -0.325, indicating a weak-to-moderate negative relationship. In this tournament, lower-rated players tended to perform better relative to their expected scores, although the relationship was not strong.

Overall, this analysis demonstrated how Elo ratings can estimate tournament performance and how actual results can differ from those expectations.

References

Glickman, M. E. *A Comprehensive Guide to Chess Ratings.* https://www.glicko.net/research/acjpaper.pdf